Weekly Report: [W04]

General Information

- **Group ID:** 52
- **Group Name:** Group 52
- **Class:** 25A01
- **Project Name:** Super Mario Game
- **Date range:** 2026-05-29 – 2026-07-04
- **Github Repository:** https://github.com/ndmhuy/SuperMarioGame.git

Tasks Completed This Week (Comprehensive Project Summary from Start)

Since this is the first weekly progress report, the following sections summarize all work completed on the project since its inception on May 29, 2026, mapping achievements across all developer branches.

25125083 - Nguyễn Đình Minh Huy (Member A - Engine, Physics & Player/Item Entities)

Branch: `main` / `dev` ***(Project Setup & Core)***

- **Task 1: Environment & Directory Setup**: Configured the CMake environment utilizing FetchContent to fetch and compile SFML 3.0.2 and Dear ImGui 1.91.8 + ImGui-SFML 3.0. Created the standard OOP project structure (`include/` , `src/` , `assets/` , `Report/`).
- **Task 2: Architectural Blueprints**: Drafted the project specifications (v2.0 expand spec outlining 110 features), UML class diagrams, and sequential implementation plan (`implementation_plan.md` , `TASKS.md` , `TASK_DIVISION.md`).
- **Task 3: Core Infrastructure Skeletons**: Provided clean C++ base skeletons for managers, physics, and entities.
- **Task 4: Math Utilities**: Implemented `MathUtils` providing vector operations, clamp, and interpolation helper methods.

- **Task 5: Game Loop & GameStateManager**: Encapsulated state management using a state stack pattern (`IGameState` interface and `GameStateManager` class), wiring in the main game loop (`Game` singleton) running at a fixed timestep (60 Hz updates) with interpolated rendering.

Branch: `A/feature/physics` ***(Collision & Physics Engine)***

- **Task 6: Collision Primitives & Broadphase**: Implemented Axis-Aligned Bounding Box (`AABB`) math and direction sensing. Developed a dynamic `SpatialHash` grid system to act as a broadphase collision pruning filter.
- **Task 7: Physics Engine Integration**: Programmed the update pipeline incorporating gravity calculations (with support for environmental gravity zones/water viscosity) and axis-separated tile resolving inside `PhysicsEngine.cpp` .
- **Task 8: Collision Resolution System**: Implemented `CollisionDetector` and `CollisionResolver` , handling solid tiles, conveyor pushes, player-vs-enemy interactions (stomps/damage), and player-vs-player versus bounces.
- **Task 9: OOP Encapsulation Safeguards**: Restructured core attributes (`position` , `velocity` , `boundingBox`) of the base `Entity` under `protected` scopes. Allowed access to `PhysicsEngine` and `CollisionResolver` via a friend-class relationship, and implemented public read-only constant reference getters to maintain encapsulation integrity.

Branch: `A/feature/entities` ***(Player & Item Entities)***

- **Task 10: Player State Pattern**: Implemented state transition updates, decorator constructors, duration timers, and unwrapping for temporary decorators (`StarDecorator` and `MegaDecorator`) in `IPlayerState.cpp` .
- **Task 11: Playable Character Mechanics**: Coded the concrete character classes (`Mario` , `Luigi` , `Toad` , `Peach`) with custom physics multipliers and special abilities (double-jump, hover flight, slide friction bypass).
- **Task 12: Item Hierarchy**: Created the base `Item` class and 12 concrete item subclasses (Mushroom, FireFlower, Coin, Star, OneUpMushroom, CapeFeather, MegaMushroom, MiniMushroom, POWBlock, PSwitch, Trampoline, StarCoin) supporting movement, bounces, collections, and EventBus notifications.

25125084 - Trần Gia Huy (Member B - Input, Sound & Gameplay Systems)

Branch: `B/feature/core-engine`

- **Task 1: InputManager (Command Pattern)**: Built the input registry mapping raw SFML 3.0 key presses and releases to action command objects (`MoveCommand` , `JumpCommand` ,

`FireCommand`, `CrouchCommand`). Supported dual-player mapping (P1: WASD/Space/F; P2: Arrows/M) and command execution queues.

- **Task 2: SoundManager (Singleton Pattern)**: Implemented audio management with separate channels for music and SFML sound effects (SFX), loading and playing WAV/OGG files. Integrated feedback features such as `resumeMusic()` and state-checks to prevent songs from restarting on transition events.

Branch: `B/feature/entities` ***(In Progress)***

- **Task 3: Enemy AI Strategies Research**: Researched and designed the `IMovementStrategy` interface and 8 AI movement strategies (Patrol, Chase, Fly, TimerEmergence, Linear, HammerThrow, TetheredChase, ProximityTrigger) for enemies.
- **Task 4: Skeletons for Enemy & Block Types**: Prepared abstract base skeletons for Enemies (Goomba, Koopa Troopa, Paratroopa, Boo, Piranha Plant, etc.) and Blocks (Brick, Question, Pipe, Hidden, Moving Platform).
- **Task 5: Registry Factory Blueprint**: Designed the layout for a config-driven `EntityFactory` registry to instantiate entities dynamically from JSON map files.

AI Usage Declaration

- Used AI to design project MVC folders, set up the SFML 3.x/ImGui CMake configuration, and troubleshoot new SFML 3.0 API changes (like `sf::Event::is<T>()` and font loading).
- Used AI to structure the State & Decorator class patterns for player power-ups and invincibility.
- Used AI to brainstorm bouncing physics vectors for items (Mushroom/Star) and float physics for Peach.
- Used AI to resolve compiler warnings regarding protected member access across class boundaries by introducing public getters and setters.

Tasks Planned for Next Week

- **Nguyễn Đình Minh Huy**: Implement Phase 4 (TileMap, LevelLoader, Level JSON files, Camera scrolling, and PlayingState integration).
- **Trần Gia Huy**: Complete and merge Phase 3 concrete enemies, blocks, and `EntityFactory` implementation, and begin Phase 5 presentation assets and animations (SpriteSheet, AnimationManager, HUD, Minimap, Particles).

Issues

- **Issue 1**: SFML 3.0 has breaking changes from SFML 2.x (e.g. `loadFromFile` replaces older texture setters, event polling uses scoped enums, and event checking is variant-based).
- **Resolution**: Consulted the official SFML 3.0 API documentation, updated the event loop, and used correct scoped enums (such as `sf::Keyboard::Key::W`).
- **Issue 2**: Moving items (Mushrooms/Stars) got stuck in tile corners because standard collision resolution cancels all velocity components along the contact normal.
- **Resolution**: Implemented custom collision check heuristics inside each item's update loop. If horizontal velocity is zeroed, the item reverses direction. For the Star, vertical velocity zeroing triggers an upward impulse.
- **Issue 3**: Encapsulated protected member variables of the player character could not be modified by interactive items (like the Trampoline spring) due to C++ visibility constraints.
- **Resolution**: Added a public `setVelocity()` method in the base `Entity` class, allowing external controllers and interactive objects to modify velocity vectors safely while preserving encapsulation of internal position/boundingBox attributes.